DATA VISUALIZATION SERIES · BEGINNER LEVEL · 2025 EDITION

Python
Visualization Libraries

A Comprehensive Documentation Guide

Matplotlib **Seaborn**

With Live Graph Examples & Code Snippets

Libraries

- Matplotlib
- Seaborn

Level

- Beginner

Graphs

- 7 chart types covered per library

Features

- Code snippets for every chart type
- Live rendered graph examples
- Side-by-side library comparison

Chart Types

14 total (7 × 2 libs)

All major plot types

S Code Snippets

Ready-to-run examples

Copy-paste friendly

C Comparison

Matplotlib vs Seaborn

Pros, cons & use cases

Table of Contents

1. Introduction
2. Matplotlib — Overview & Graph Types
2.1 Line Plot
2.2 Scatter Plot
2.3 Bar Chart
2.4 Histogram
2.5 Pie Chart
2.6 Box Plot
2.7 Heatmap
3. Seaborn — Overview & Graph Types
3.1 Scatter / Relational Plot
3.2 Line Plot with CI
3.3 Bar Plot
3.4 Histogram + KDE
3.5 Box Plot
3.6 Violin Plot
3.7 Heatmap
4. Comparison: Matplotlib vs Seaborn
5. Resources & References

1. Introduction

Python's visualization ecosystem is one of the richest in any programming language. From simple line charts to complex interactive dashboards, Python provides tools for every level of data storytelling. This guide focuses on **Matplotlib** and **Seaborn** — the two most foundational and widely adopted libraries — covering 7 chart types each, complete with live rendered graph examples, ready-to-run code snippets, and a detailed side-by-side comparison.

Whether you are a student exploring data for the first time, a data analyst building exploratory reports, or a researcher preparing publication-quality figures, understanding these two libraries gives you a solid foundation to work with any visualization tool in the Python ecosystem.

Why Data Visualization Matters

Raw numbers alone rarely communicate insight. Visualization transforms datasets into patterns, trends, and outliers that the human eye can immediately grasp. Key benefits include:

- **Pattern Recognition:** Spot trends, seasonality, and correlations at a glance that would be invisible in a table of numbers.
- **Outlier Detection:** Box plots, scatter plots, and histograms surface anomalous data points that may indicate errors or important findings.
- **Communication:** A well-crafted chart conveys results to non-technical stakeholders faster and more memorably than any written summary.
- **Model Evaluation:** Loss curves, confusion-matrix heatmaps, and ROC plots are indispensable for diagnosing machine-learning models.
- **Exploratory Data Analysis (EDA):** Visualizing distributions, pairwise relationships, and missing values guides feature engineering decisions.

Matplotlib — Foundational Plotting Library

Created by John D. Hunter in 2003 and inspired by MATLAB's plotting interface, Matplotlib is the bedrock of Python visualization. Almost every other plotting library — including Seaborn, Pandas .plot(), and Plotly — is either built on top of or heavily influenced by Matplotlib.

Key Characteristics

- **Type:** Low-level, imperative, static
- **Installation:** `pip install matplotlib`
- **Primary Interface:** `matplotlib.pyplot` (MATLAB-like) and the object-oriented `Figure / Axes` API
- **Output Formats:** PNG, SVG, PDF, EPS — plus interactive backends (Qt, Tk, WebAgg)
- **Integration:** NumPy arrays, Pandas Series/DataFrames, SciPy, Jupyter Notebooks
- **Best For:** Full control over every visual element, custom multi-panel layouts, animations, and publication-quality scientific figures

2. Matplotlib

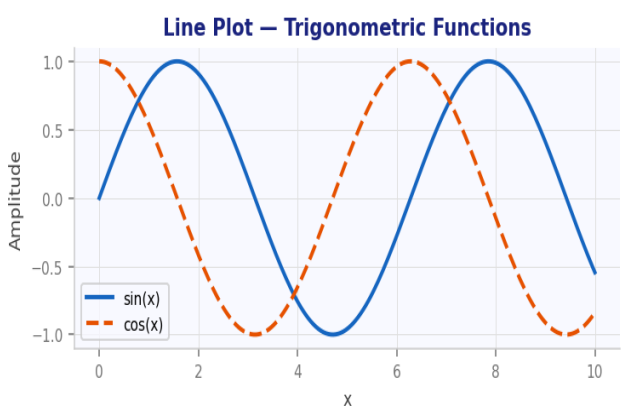
Matplotlib is the foundational plotting library in Python, built by John D. Hunter in 2003. It provides an object-oriented API and a MATLAB-like **pyplot** interface. Nearly every other Python visualization library is built on or inspired by Matplotlib.

Feature	Detail
Installation	pip install matplotlib
Import	import matplotlib.pyplot as plt
Output	Static (PNG, SVG, PDF) + interactive backends
Integration	NumPy, Pandas, SciPy, Jupyter

2.1 Line Plot

Description: Connects data points with a continuous line. Ideal for visualizing trends over a continuous variable.

Use Case: Stock prices over time, temperature trends, model training loss curves.

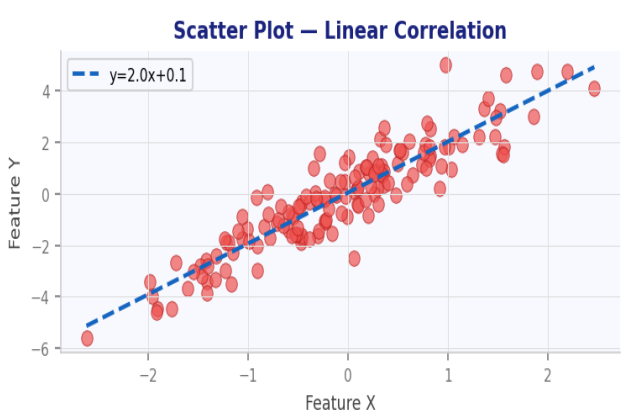



```
import matplotlib.pyplot as plt import numpy
as np x = np.linspace(0, 10, 200)
plt.figure(figsize=(8, 4)) plt.plot(x,
np.sin(x), color='steelblue', lw=2,
label='sin(x)') plt.plot(x, np.cos(x),
color='#e65100', lw=2, ls='--',
label='cos(x)') plt.title('Line Plot — Sine
& Cosine') plt.xlabel('X');
plt.ylabel('Amplitude') plt.legend();
plt.grid(True, alpha=0.4)
plt.tight_layout(); plt.show()
```

2.2 Scatter Plot

Description: Plots individual data points to reveal correlations, clusters, or outliers.

Use Case: Correlation analysis, cluster visualization, outlier detection.

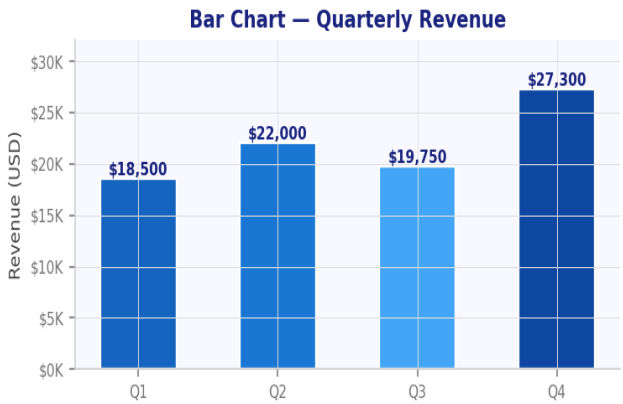



```
import matplotlib.pyplot as plt import numpy
as np np.random.seed(42) x =
np.random.randn(150) y = 2 * x +
np.random.randn(150) * 0.8 plt.scatter(x, y,
c='coral', edgecolors='darkred', alpha=0.7,
s=60) m, b = np.polyfit(x, y, 1)
plt.plot(np.sort(x), m*np.sort(x)+b,
color='steelblue', lw=2, ls='--')
plt.title('Scatter Plot — Correlation')
plt.tight_layout(); plt.show()
```

2.3 Bar Chart

Description: Uses rectangular bars to compare discrete categories by value.

Use Case: Comparing sales by quarter, survey results, frequency counts.

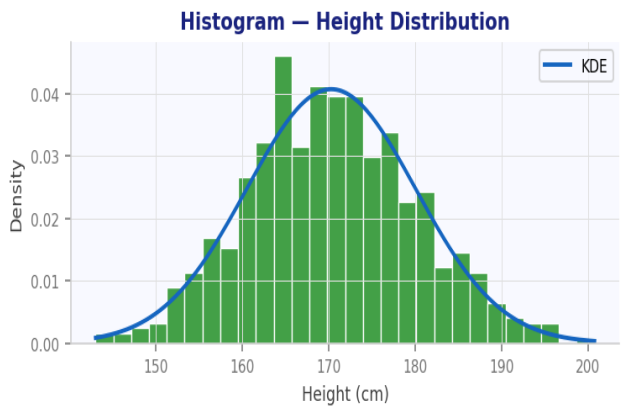



```
import matplotlib.pyplot as plt categories =
['Q1', 'Q2', 'Q3', 'Q4'] values = [18500,
22000, 19750, 27300] plt.figure(figsize=(7,
4)) bars = plt.bar(categories, values,
color='#1565c0', edgecolor='white',
linewidth=1.2) for bar, val in zip(bars,
values):
plt.text(bar.get_x()+bar.get_width()/2,
bar.get_height()+300, f'${val:,}',
ha='center', fontsize=9) plt.title('Bar
Chart — Quarterly Revenue')
plt.tight_layout(); plt.show()
```

2.4 Histogram

Description: Groups continuous data into bins and shows the frequency of each bin.

Use Case: Understanding data distribution, checking normality, visualizing age groups.

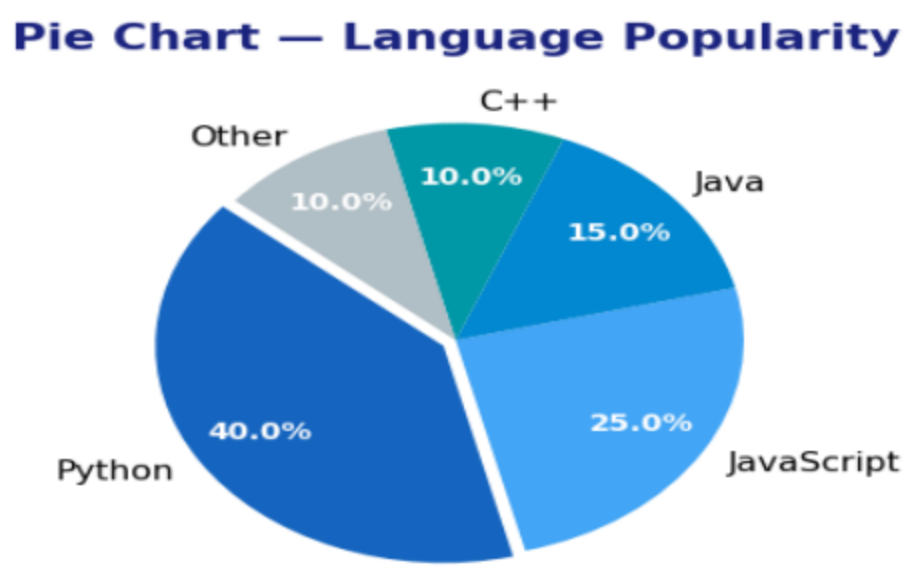



```
import matplotlib.pyplot as plt import numpy
as np from scipy.stats import norm data =
np.random.normal(loc=170, scale=10,
size=600) plt.figure(figsize=(7, 4))
plt.hist(data, bins=28,
color='mediumseagreen', edgecolor='white',
density=True) xc = np.linspace(data.min(),
data.max(), 200) plt.plot(xc, norm.pdf(xc,
data.mean(), data.std()), color='steelblue',
lw=2, label='KDE') plt.title('Histogram —
Height Distribution') plt.tight_layout();
plt.show()
```

2.5 Pie Chart

Description: Shows proportional parts of a whole as slices of a circle.

Use Case: Market share, budget allocation, survey response breakdown.

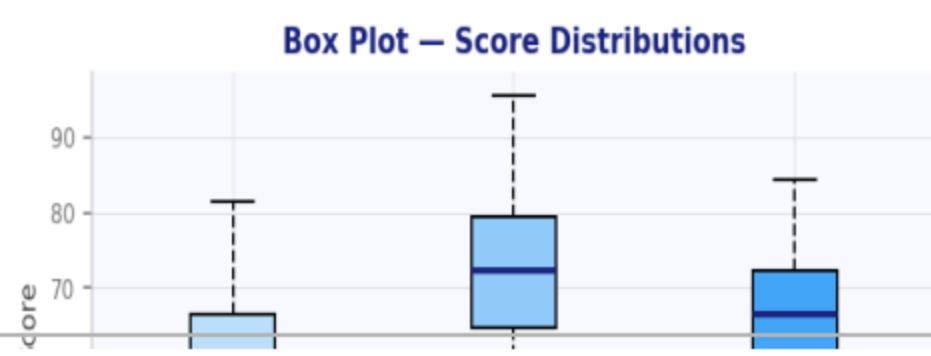



```
import matplotlib.pyplot as plt labels =
['Python','JavaScript','Java','C++','Other']
sizes = [40, 25, 15, 10, 10] colors = ['#156
5c0','#42a5f5','#0288d1','#0097a7','#b0bec5'
] explode = (0.05, 0, 0, 0, 0)
plt.figure(figsize=(6, 5)) plt.pie(sizes,
explode=explode, labels=labels,
colors=colors, autopct='%1.1f%%',
startangle=140, shadow=True) plt.title('Pie
Chart — Language Popularity')
plt.tight_layout(); plt.show()
```

2.6 Box Plot

Description: Summarizes distribution using quartiles (Q1, median, Q3) and highlights outliers.

Use Case: Comparing score distributions across groups, spotting outliers.

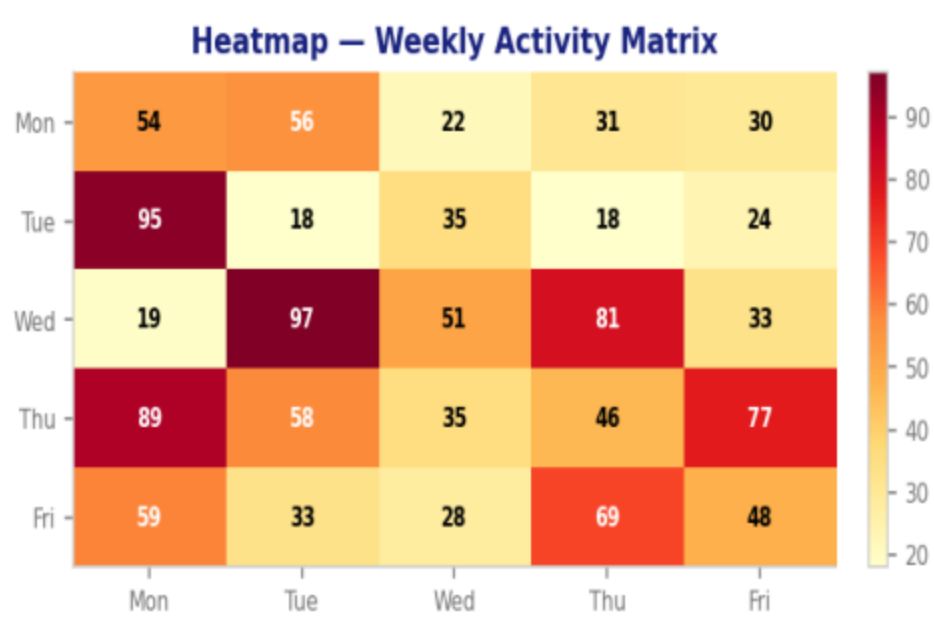



```
import matplotlib.pyplot as plt import numpy
as np data = [np.random.normal(60, 10, 100),
np.random.normal(70, 12, 100),
np.random.normal(65, 8, 100)] fig, ax =
plt.subplots(figsize=(7, 4)) bp =
ax.boxplot(data, patch_artist=True,
tick_labels=['A','B','C']) for patch, c in
```

2.7 Heatmap (via imshow)

Description: Encodes matrix values as colors in a grid. Useful for correlation matrices and 2-D density maps.

Use Case: Correlation analysis, confusion matrices, geographic activity.




```
import matplotlib.pyplot as plt import numpy
as np matrix = np.random.randint(0, 100, (5,
5)) days = ['Mon','Tue','Wed','Thu','Fri']
fig, ax = plt.subplots(figsize=(5, 4)) im =
ax.imshow(matrix, cmap='YlOrRd')
plt.colorbar(im, ax=ax)
ax.set_xticks(range(5));
ax.set_xticklabels(days)
ax.set_yticks(range(5));
ax.set_yticklabels(days) for i in range(5):
for j in range(5): ax.text(j, i,
matrix[i,j], ha='center', va='center',
fontsize=9) ax.set_title('Heatmap — Weekly
Activity') plt.tight_layout(); plt.show()
```

3. Seaborn

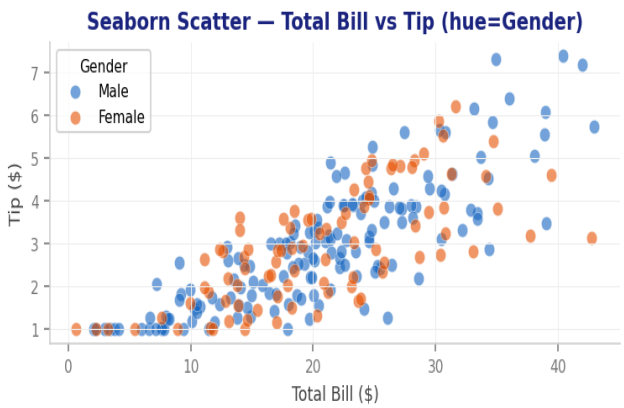
Seaborn is a statistical visualization library built on Matplotlib, developed by Michael Waskom. It provides a high-level interface for drawing attractive statistical graphics and treats Pandas DataFrames as first-class citizens.

Feature	Detail
Installation	pip install seaborn
Import	import seaborn as sns
Output	Static (uses Matplotlib backend)
Integration	Pandas DataFrames, Matplotlib, SciPy
Theme System	sns.set_theme(), set_style(), set_palette()

3.1 Scatter / Relational Plot

Description: sns.scatterplot() plots two continuous variables with optional hue, size, and style encodings.

Use Case: Exploring feature relationships, multi-dimensional EDA on structured data.

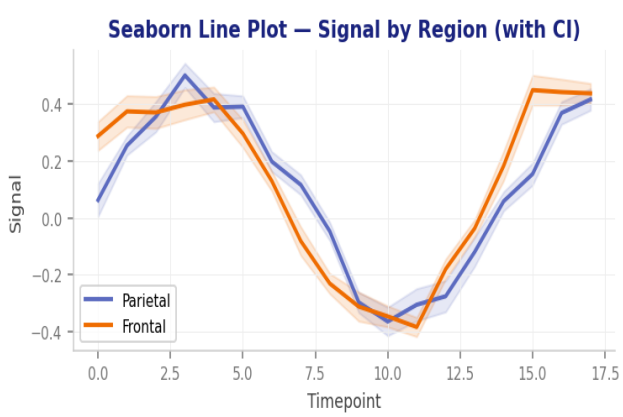



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='whitegrid') tips =
sns.load_dataset('tips')
plt.figure(figsize=(7, 5))
sns.scatterplot(data=tips, x='total_bill',
y='tip', hue='sex', size='size', sizes=(40,
200), palette='Set2', alpha=0.8)
plt.title('Total Bill vs Tip — by Gender')
plt.tight_layout(); plt.show()
```

3.2 Line Plot with CI

Description: sns.lineplot() automatically computes confidence intervals and aggregates repeated measurements.

Use Case: Time-series trends with uncertainty, model performance curves, A/B test results.

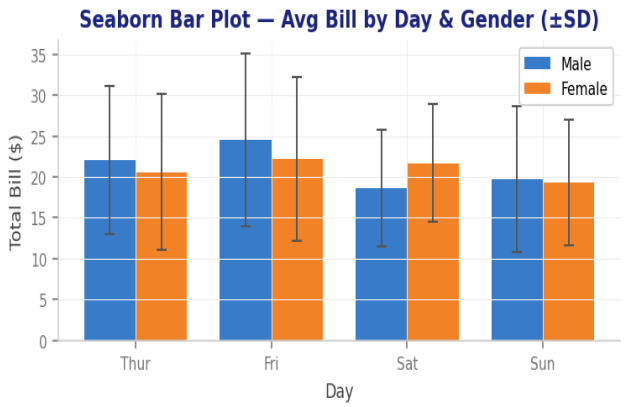



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='darkgrid') fmri =
sns.load_dataset('fmri')
plt.figure(figsize=(8, 4))
sns.lineplot(data=fmri, x='timepoint',
y='signal', hue='region', style='event',
palette='muted', linewidth=2)
plt.title('fMRI Signal Over Time by Region')
plt.tight_layout(); plt.show()
```

3.3 Bar Plot

Description: sns.barplot() shows the mean of a numeric variable per category with bootstrapped confidence intervals.

Use Case: Comparing group means, A/B testing results, category benchmarks.

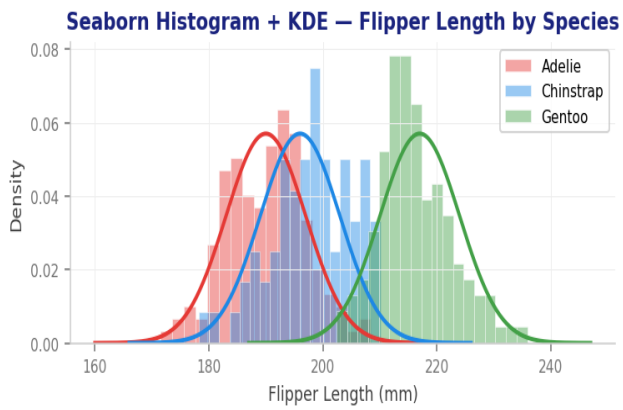



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='whitegrid') tips =
sns.load_dataset('tips')
plt.figure(figsize=(6, 4))
sns.barplot(data=tips, x='day',
y='total_bill', hue='sex', palette='pastel',
errorbar='sd',
order=['Thur','Fri','Sat','Sun'])
plt.title('Avg Total Bill by Day and
Gender') plt.tight_layout(); plt.show()
```

3.4 Histogram + Distribution Plot

Description: sns.histplot() combines histograms with optional KDE curves for richer distribution views.

Use Case: Visualizing distributions, comparing groups, normality checks.

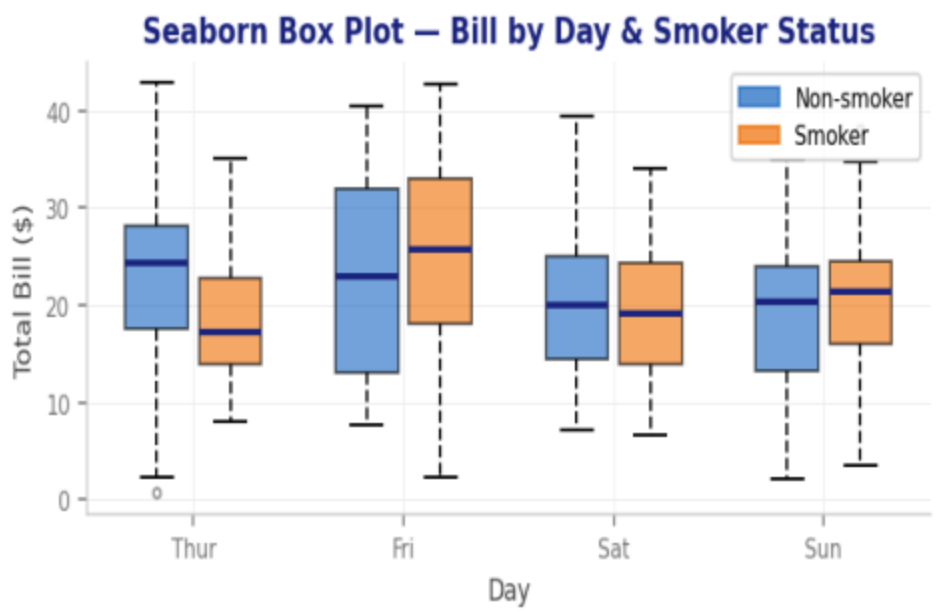



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='ticks') penguins =
sns.load_dataset('penguins')
plt.figure(figsize=(7, 4))
sns.histplot(data=penguins,
x='flipper_length_mm', hue='species',
kde=True, palette='Set1', bins=22,
alpha=0.6) plt.title('Flipper Length
Distribution') plt.tight_layout();
plt.show()
```

3.5 Box Plot

Description: sns.boxplot() provides richly styled box plots with automatic hue grouping.

Use Case: Comparing distributions, spotting outliers across multiple categories.

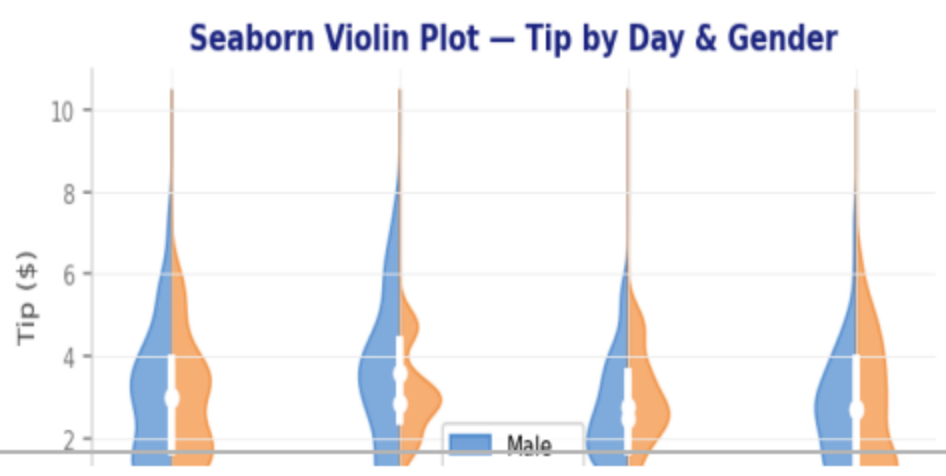



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='whitegrid',
palette='pastel') tips =
sns.load_dataset('tips')
plt.figure(figsize=(7, 4))
sns.boxplot(data=tips, x='day',
y='total_bill', hue='smoker',
order=['Thur','Fri','Sat','Sun'])
plt.title('Bill Distribution by Day &
Smoker') plt.tight_layout(); plt.show()
```

3.6 Violin Plot

Description: sns.violinplot() combines a box plot with KDE, showing both summary stats and distribution shape.

Use Case: Comparing distributions in detail, bimodal data, publication figures.

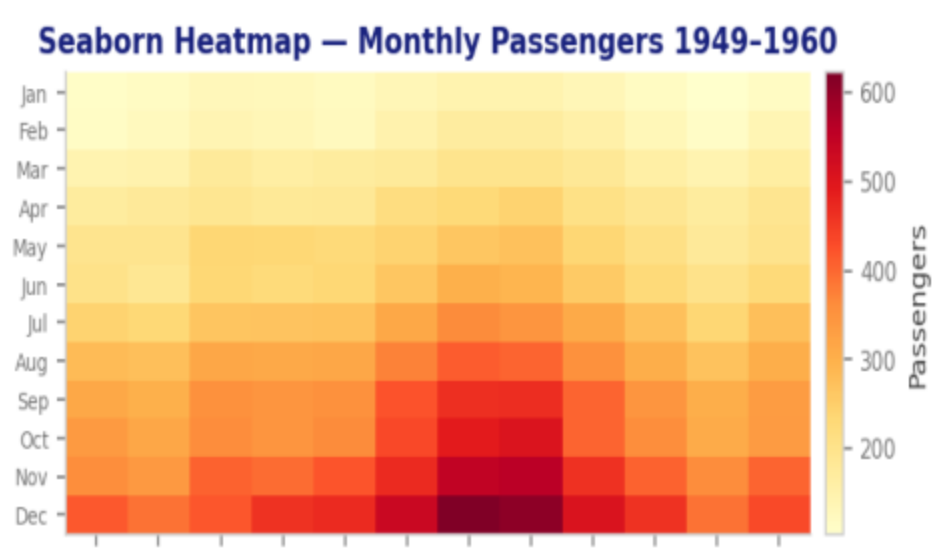



```
import seaborn as sns import
matplotlib.pyplot as plt
sns.set_theme(style='whitegrid') tips =
sns.load_dataset('tips')
plt.figure(figsize=(7, 4))
sns.violinplot(data=tips, x='day', y='tip',
hue='sex', split=True, inner='quart',
palette={'Male':'#4472c4',
'Female':'#ed7d31'},
order=['Thur','Fri','Sat','Sun'])
```

3.7 Heatmap

Description: sns.heatmap() renders a DataFrame or 2-D array as a colored grid with built-in annotations.

Use Case: Correlation matrices, pivot tables, missing-value maps.




```
import seaborn as sns import
matplotlib.pyplot as plt sns.set_theme()
flights = sns.load_dataset('flights') pivot
= flights.pivot_table( index='month',
columns='year', values='passengers')
plt.figure(figsize=(10, 5))
sns.heatmap(pivot, annot=True, fmt='d',
cmap='YlOrRd', linewidths=0.4,
linecolor='white') plt.title('Monthly
Passengers 1949-1960') plt.tight_layout();
plt.show()
```

4. Comparison: Matplotlib vs Seaborn

Both libraries serve the same purpose but excel in different scenarios. The table below summarizes key differences across 10 dimensions:

Dimension	Matplotlib	Seaborn
Ease of Use	Steeper learning curve; explicit styling for every element.	Beginner-friendly; polished plots with minimal code.
Code Verbosity	More lines; explicit figure/axes management.	Concise; grouping, stats, aesthetics in one call.
Default Aesthetics	Plain defaults; customization required.	Beautiful defaults via themes (whitegrid, ticks…).
Statistical Tools	None built-in; manual calculation required.	Auto CI bands, KDE, regression, aggregation built in.
Customization	Full control over every element.	Good via Matplotlib; overriding defaults can be tricky.
Interactivity	Basic zoom/pan via GUI backends.	Same as Matplotlib — static by default.
Large Datasets	Handles large arrays well; fast rendering.	Same performance; Pandas groupby adds overhead.
Pandas Support	Basic; pass Series/arrays manually.	Native; data= accepts DataFrame + column names.
Learning Curve	Moderate; many concepts (Figure, Axes, Artists).	Gentle; higher abstractions, less setup needed.
When to Use	Custom layouts, subplots, animations, full control.	EDA, statistics, categorical plots, quick visuals.

Quick Decision Guide: Use **Seaborn** for fast, attractive statistical plots from a Pandas DataFrame with minimal code. Use **Matplotlib** when you need fine-grained control, custom multi-panel layouts, animations, or plot types not supported by higher-level libraries.

5. Resources & References

Resource	URL	Notes
Matplotlib Quick Start	`https://matplotlib.org/stable/users/explain/quick_start.html`	Official quick-start: Figure, Axes, basic plot types.
Matplotlib Gallery	`https://matplotlib.org/stable/gallery/`	Hundreds of examples with source code.
Seaborn Introduction	`https://seaborn.pydata.org/tutorial/introduction.html`	Official tutorial: themes, datasets, core plots.
Seaborn API Reference	`https://seaborn.pydata.org/api.html`	Complete function reference.
Plotly Python	`https://plotly.com/python/`	Interactive charts — great next step after static plots.
Bokeh User Guide	`https://docs.bokeh.org/en/latest/docs/user_guide/basic.html`	Web-interactive plotting; glyphs, tools, server apps.
Pandas Visualization	`https://pandas.pydata.org/docs/user_guide/visualization.html`	Built-in .plot() interface for quick DataFrame charts.

Suggested Learning Path

Once comfortable with these two libraries, explore:

- **Plotly** — Interactive web charts with 3D, maps, and dashboards.
- **Bokeh** — Browser-based interactive plots with JavaScript widgets.
- **Altair** — Declarative visualization based on Vega-Lite grammar.
- **Folium** — Interactive geographic maps built on Leaflet.js.